ECEN 260 - Laboratory **Servo Temperature Gauge**

Sashin Shim

Instructor: Brother Allred
Date: 12/16/2025
BYU-Idaho Semester: Fall 2025

Contents

1 Lab Overview **3**
1.1 Objectives 3

2 Specifications **4**
2.1 Operating Procedures 4
2.2 Known Limitations 4
2.3 Parts List 4

3 Schematics **6**

4 Test Plan and Test Results **7**

5 Code **9**
5.1 Code selection from main.c 9

6 Visuals **21**

7 Conclusion **22**

List of Figures

1 Servo Gauge Schematic) 6
2 Replace this text with a caption for your image.) 21

1 Lab Overview

The goal of this lab was to make a temperature monitoring system using an STM32. The board reads temperature from a thermistor and then reacts to it in a few different ways. A servo moves like a gauge to show how hot it is, LEDs show whether the temperature is okay or getting too high, and a buzzer turns on if things get bad. The point was to get all the parts working together instead of just testing one thing at a time.

This lab is based on a real situation where electronics can overheat, like power connectors or components pulling too much current. The system is meant to warn you before things get dangerous. I focused more on making it actually work and be easy to understand rather than making it overly complicated.

1.1 Objectives

- Read temperature using a thermistor and the ADC
- Use PWM and a timer to control a servo motor
- Control LEDs using GPIO to show temperature ranges
- Use a buzzer to give an audible warning

2 Specifications

This project is a temperature monitoring and warning system built using an STM32 microcontroller. The system continuously reads temperature from a thermistor using the ADC and reacts based on predefined temperature ranges. A servo motor acts as a visual gauge that moves according to temperature level. Three LEDs indicate the current temperature state, with green meaning safe, yellow meaning warning, and red meaning overheating. A buzzer provides an audible alert when the temperature reaches warning or danger levels. The system is designed to simulate monitoring overheating in electronics, such as power connectors or components under heavy load.

The system runs automatically once powered on and does not require user input during normal operation. All outputs update in real time based on the sensor reading. The design focuses on being easy to understand and clearly showing when temperature becomes unsafe.

2.1 Operating Procedures

- Power the STM32 board using USB or an external power source.
- Ensure the thermistor is connected properly with a resistor forming a voltage divider.
- Once powered, the system begins reading temperature automatically.
- Observe the servo position to see the current temperature level.
- Watch the LEDs to determine if the temperature is safe, warning, or dangerous.
- Listen for the buzzer, which activates in higher temperature ranges.

2.2 Known Limitations

- The system does not display an exact temperature value, only relative ranges.
- Temperature accuracy depends on the thermistor and resistor values used.
- The system is designed for demonstration purposes and not long term industrial use.
- No data logging or communication to a computer is implemented.

2.3 Parts List

- STM32 Nucleo development board
- Thermistor
- Fixed resistor for voltage divider
- Servo motor
- Three LEDs

- Buzzer
- Breadboard and jumper wires

3 Schematics

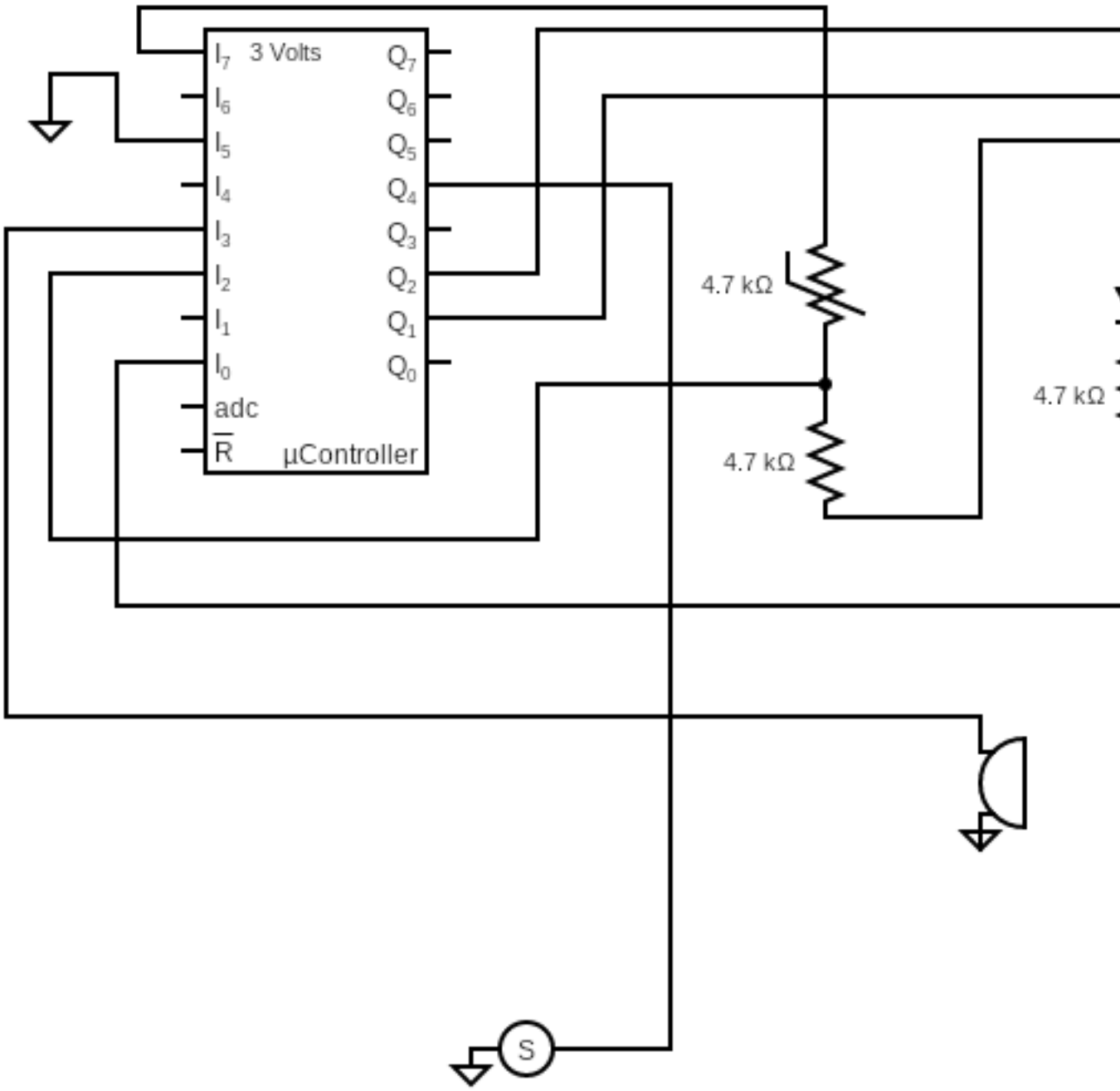

Figure 1: Servo Gauge Schematic)

4 Test Plan and Test Results

The test plan for this project focuses on making sure the system responds correctly to different temperature levels and wiring conditions. Testing was done by changing the input to the thermistor and observing how the servo, LEDs, and buzzer reacted. Tests included normal operating conditions, edge cases near the temperature thresholds, and error cases such as incorrect wiring or disconnected components. The goal was to confirm that the system behaves in a clear and predictable way and gives proper warnings as temperature increases.

- Test Scenario #1 Normal temperature range
  - Step 1: Power on the STM32 board.
  - Step 2: Set the thermistor to a low temperature condition.
  - Step 3: Observe the servo position, LEDs, and buzzer.
  - Expected result: Green LED on, servo near the safe position, buzzer off.
  - Actual result: Green LED turned on, servo stayed in the safe range, buzzer remained off.
- Test Scenario #2 Warning temperature range
  - Step 1: Increase the temperature applied to the thermistor.
  - Step 2: Allow the ADC reading to enter the warning range.
  - Step 3: Observe LED, servo movement, and buzzer behavior.
  - Expected result: Yellow LED on, servo moves toward higher range, buzzer beeps slowly.
  - Actual result: Yellow LED turned on, servo moved as expected, buzzer beeped at a slow rate.
- Test Scenario #3 Over temperature range
  - Step 1: Increase the thermistor temperature to a high level.
  - Step 2: Wait for the system to update.
  - Step 3: Observe outputs.
  - Expected result: Red LED on, servo at maximum position, buzzer beeps rapidly.
  - Actual result: Red LED turned on, servo moved to the maximum position, buzzer beeped rapidly.
- Test Scenario #4 Edge case at threshold values
  - Step 1: Slowly adjust the thermistor near the green to yellow boundary.
  - Step 2: Observe transitions between LEDs.
  - Step 3: Watch for unexpected behavior.

  - Expected result: LEDs change cleanly at threshold values without flickering.
  - Actual result: LEDs switched cleanly at the thresholds with no flicker.

- Test Scenario #5 Error case incorrect or loose wiring
  - Step 1: Disconnect or loosen the thermistor wiring.
  - Step 2: Power the system and observe behavior.
  - Step 3: Reconnect wiring properly.
  - Expected result: System shows abnormal readings indicating a fault condition.
  - Actual result: System entered the red state, making the wiring issue obvious and easy to diagnose.

5 Code

5.1.

5.1 Code selection from main.c

```c
/* USER CODE BEGIN Header */
/**
 ******************************************************************************
 * @file           : main.c
 * @brief          : Main program body
 ******************************************************************************
 * @attention
 *
 * Copyright (c) 2025 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 ******************************************************************************
 */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------ */
#include "main.h"

/* Private includes ---------------------------------------------------------- */
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Private typedef ----------------------------------------------------------- */
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------ */
/* USER CODE BEGIN PD */

/* USER CODE END PD */

```

```c
/* Private macro -------------------------------------------------------------
   */
/* USER CODE BEGIN PM */
#define green_min 1600
#define green_max 1700
#define yellow_max 1800
#define red_max 1900
/* USER CODE END PM */

/* Private variables ---------------------------------------------------------
   */
ADC_HandleTypeDef hadc1;

I2C_HandleTypeDef hi2c1;
I2C_HandleTypeDef hi2c2;

TIM_HandleTypeDef htim2;

UART_HandleTypeDef huart2;

/* USER CODE BEGIN PV */

/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------
   */
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USART2_UART_Init(void);
static void MX_ADC1_Init(void);
static void MX_I2C1_Init(void);
static void MX_I2C2_Init(void);
static void MX_TIM2_Init(void);
/* USER CODE BEGIN PFP */


/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------
   */
/* USER CODE BEGIN 0 */
uint32_t lastBeep = 0;
uint8_t buzzerState = 0;
// Function for controlling the Servo Motor (TIM2 Channel 2 on PB3)
// Angle range: 0 to 180 degrees.
void Set_Servo_Angle(int32_t angle)
{
    // Clamp angle between 0 and 180
    if (angle > 180) angle = 180;
    if(angle < 0) angle = 0;

    // Calculate the pulse value (CCR)
    // Pulse = 500 (0.5ms) + (Angle * 2000 / 180)
    uint32_t pulse = 500 + (angle * 2000 / 180);
```

```
    // Set the Pulse for PWM Channel 2
    __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, pulse);
}


/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{

  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------
   */

  /* Reset of all peripherals, Initializes the Flash interface and the Systick
   . */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_USART2_UART_Init();
  MX_ADC1_Init();
  MX_I2C1_Init();
  MX_I2C2_Init();
  MX_TIM2_Init();
  /* USER CODE BEGIN 2 */
 HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1); // start pwm for servo
  /* Optionally init LCD if using driver */
#ifdef USE_LCD
  lcd_init(&hi2c1);
  lcd_setCursor(0,0);
  lcd_print("Boot...");
  #endif
  /* USER CODE END 2 */

```

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
/* USER CODE BEGIN WHILE */
while (1) { // LED control loop//
  HAL_ADC_Start(&hadc1);
  HAL_ADC_PollForConversion(&hadc1, HAL_MAX_DELAY);
  uint32_t adc_val = HAL_ADC_GetValue(&hadc1); // 0  4095
  int32_t adc_capped = adc_val;
  if(adc_capped < green_min){
    adc_capped = green_min;
  }
  else if(adc_capped > red_max){
    adc_capped = red_max;
  }
  int32_t servo_angle = 180-(((adc_capped - green_min)*180) /(red_max -
 green_min));
  // --- 3. Control Servo Motor ---
  Set_Servo_Angle(servo_angle);


  // TEMP: light LEDs according to ADC value
  HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0 | GPIO_PIN_1 | GPIO_PIN_2,
      GPIO_PIN_RESET); // turn all off
  if (adc_val < green_max)
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0, GPIO_PIN_SET);  // green
  else if (adc_val < yellow_max)
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_1, GPIO_PIN_SET);  // yellow
  else
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_2, GPIO_PIN_SET);  // red

  HAL_Delay(100); // small delay so LEDs are visible

  // For the buzzer
  uint32_t now = HAL_GetTick();
  uint32_t beepInterval = 0;

  if (adc_val < green_max) {
    //green should be off
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_10, GPIO_PIN_RESET);

  } else if (adc_val < yellow_max) {
    // yellow should be on and beep slowly
    beepInterval = 500;
  }
  else {
    //red should be on beeping fast
    beepInterval = 100;
  }
  if (beepInterval > 0) {
    if (now - lastBeep >= beepInterval) {
      lastBeep = now;
      buzzerState = !buzzerState;
      HAL_GPIO_WritePin(GPIOB, GPIO_PIN_10, buzzerState);
    }
```

```
    }
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}

/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

  /** Configure the main internal regulator output voltage
  */
  if (HAL_PWREx_ControlVoltageScaling(PWR_REGULATOR_VOLTAGE_SCALE1) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
  RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSI;
  RCC_OscInitStruct.PLL.PLLM = 1;
  RCC_OscInitStruct.PLL.PLLN = 10;
  RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV7;
  RCC_OscInitStruct.PLL.PLLQ = RCC_PLLQ_DIV2;
  RCC_OscInitStruct.PLL.PLLR = RCC_PLLR_DIV2;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_4) != HAL_OK)
  {
    Error_Handler();
```

```
  }
}

/**
  * @brief ADC1 Initialization Function
  * @param None
  * @retval None
  */
static void MX_ADC1_Init(void)
{

  /* USER CODE BEGIN ADC1_Init 0 */

  /* USER CODE END ADC1_Init 0 */

  ADC_MultiModeTypeDef multimode = {0};
  ADC_ChannelConfTypeDef sConfig = {0};

  /* USER CODE BEGIN ADC1_Init 1 */

  /* USER CODE END ADC1_Init 1 */

  /** Common config
  */
  hadc1.Instance = ADC1;
  hadc1.Init.ClockPrescaler = ADC_CLOCK_ASYNC_DIV1;
  hadc1.Init.Resolution = ADC_RESOLUTION_12B;
  hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
  hadc1.Init.ScanConvMode = ADC_SCAN_DISABLE;
  hadc1.Init.EOCSelection = ADC_EOC_SINGLE_CONV;
  hadc1.Init.LowPowerAutoWait = DISABLE;
  hadc1.Init.ContinuousConvMode = DISABLE;
  hadc1.Init.NbrOfConversion = 1;
  hadc1.Init.DiscontinuousConvMode = DISABLE;
  hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START;
  hadc1.Init.ExternalTrigConvEdge = ADC_EXTERNALTRIGCONVEDGE_NONE;
  hadc1.Init.DMAContinuousRequests = DISABLE;
  hadc1.Init.Overrun = ADC_OVR_DATA_PRESERVED;
  hadc1.Init.OversamplingMode = DISABLE;
  if (HAL_ADC_Init(&hadc1) != HAL_OK)
  {
    Error_Handler();
  }

  /** Configure the ADC multi-mode
  */
  multimode.Mode = ADC_MODE_INDEPENDENT;
  if (HAL_ADCEx_MultiModeConfigChannel(&hadc1, &multimode) != HAL_OK)
  {
    Error_Handler();
  }

  /** Configure Regular Channel
  */
```

```
  sConfig.Channel = ADC_CHANNEL_5;
  sConfig.Rank = ADC_REGULAR_RANK_1;
  sConfig.SamplingTime = ADC_SAMPLETIME_2CYCLES_5;
  sConfig.SingleDiff = ADC_SINGLE_ENDED;
  sConfig.OffsetNumber = ADC_OFFSET_NONE;
  sConfig.Offset = 0;
  if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN ADC1_Init 2 */

  /* USER CODE END ADC1_Init 2 */

}

/**
  * @brief I2C1 Initialization Function
  * @param None
  * @retval None
  */
static void MX_I2C1_Init(void)
{

  /* USER CODE BEGIN I2C1_Init 0 */

  /* USER CODE END I2C1_Init 0 */

  /* USER CODE BEGIN I2C1_Init 1 */

  /* USER CODE END I2C1_Init 1 */
  hi2c1.Instance = I2C1;
  hi2c1.Init.Timing = 0x10D19CE4;
  hi2c1.Init.OwnAddress1 = 0;
  hi2c1.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
  hi2c1.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
  hi2c1.Init.OwnAddress2 = 0;
  hi2c1.Init.OwnAddress2Masks = I2C_OA2_NOMASK;
  hi2c1.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
  hi2c1.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
  if (HAL_I2C_Init(&hi2c1) != HAL_OK)
  {
    Error_Handler();
  }

  /** Configure Analogue filter
  */
  if (HAL_I2CEx_ConfigAnalogFilter(&hi2c1, I2C_ANALOGFILTER_ENABLE) != HAL_OK)
  {
    Error_Handler();
  }

  /** Configure Digital filter
  */
```

```
  if (HAL_I2CEx_ConfigDigitalFilter(&hi2c1, 0) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN I2C1_Init 2 */

  /* USER CODE END I2C1_Init 2 */

}

/**
  * @brief I2C2 Initialization Function
  * @param None
  * @retval None
  */
static void MX_I2C2_Init(void)
{

  /* USER CODE BEGIN I2C2_Init 0 */

  /* USER CODE END I2C2_Init 0 */

  /* USER CODE BEGIN I2C2_Init 1 */

  /* USER CODE END I2C2_Init 1 */
  hi2c2.Instance = I2C2;
  hi2c2.Init.Timing = 0x10D19CE4;
  hi2c2.Init.OwnAddress1 = 0;
  hi2c2.Init.AddressingMode = I2C_ADDRESSINGMODE_7BIT;
  hi2c2.Init.DualAddressMode = I2C_DUALADDRESS_DISABLE;
  hi2c2.Init.OwnAddress2 = 0;
  hi2c2.Init.OwnAddress2Masks = I2C_OA2_NOMASK;
  hi2c2.Init.GeneralCallMode = I2C_GENERALCALL_DISABLE;
  hi2c2.Init.NoStretchMode = I2C_NOSTRETCH_DISABLE;
  if (HAL_I2C_Init(&hi2c2) != HAL_OK)
  {
    Error_Handler();
  }

  /** Configure Analogue filter
  */
  if (HAL_I2CEx_ConfigAnalogFilter(&hi2c2, I2C_ANALOGFILTER_ENABLE) != HAL_OK)
  {
    Error_Handler();
  }

  /** Configure Digital filter
  */
  if (HAL_I2CEx_ConfigDigitalFilter(&hi2c2, 0) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN I2C2_Init 2 */

```

```
  /* USER CODE END I2C2_Init 2 */

}

/**
  * @brief TIM2 Initialization Function
  * @param None
  * @retval None
  */
static void MX_TIM2_Init(void)
{

  /* USER CODE BEGIN TIM2_Init 0 */

  /* USER CODE END TIM2_Init 0 */

  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_OC_InitTypeDef sConfigOC = {0};

  /* USER CODE BEGIN TIM2_Init 1 */

  /* USER CODE END TIM2_Init 1 */
  htim2.Instance = TIM2;
  htim2.Init.Prescaler = 79;
  htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim2.Init.Period = 19999;
  htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
  if (HAL_TIM_PWM_Init(&htim2) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sConfigOC.OCMode = TIM_OCMODE_PWM1;
  sConfigOC.Pulse = 1500;
  sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
  sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
  if (HAL_TIM_PWM_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_1) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM2_Init 2 */

  /* USER CODE END TIM2_Init 2 */
  HAL_TIM_MspPostInit(&htim2);

}

/**
```

```
  * @brief USART2 Initialization Function
  * @param None
  * @retval None
  */
static void MX_USART2_UART_Init(void)
{

  /* USER CODE BEGIN USART2_Init 0 */

  /* USER CODE END USART2_Init 0 */

  /* USER CODE BEGIN USART2_Init 1 */

  /* USER CODE END USART2_Init 1 */
  huart2.Instance = USART2;
  huart2.Init.BaudRate = 115200;
  huart2.Init.WordLength = UART_WORDLENGTH_8B;
  huart2.Init.StopBits = UART_STOPBITS_1;
  huart2.Init.Parity = UART_PARITY_NONE;
  huart2.Init.Mode = UART_MODE_TX_RX;
  huart2.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart2.Init.OverSampling = UART_OVERSAMPLING_16;
  huart2.Init.OneBitSampling = UART_ONE_BIT_SAMPLE_DISABLE;
  huart2.AdvancedInit.AdvFeatureInit = UART_ADVFEATURE_NO_INIT;
  if (HAL_UART_Init(&huart2) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN USART2_Init 2 */

  /* USER CODE END USART2_Init 2 */

}

/**
  * @brief GPIO Initialization Function
  * @param None
  * @retval None
  */
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
  /* USER CODE BEGIN MX_GPIO_Init_1 */

  /* USER CODE END MX_GPIO_Init_1 */

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOH_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
```

```
  HAL_GPIO_WritePin(GPIOB, GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_10, GPIO_PIN_RESET);

  /*Configure GPIO pin : B1_Pin */
  GPIO_InitStruct.Pin = B1_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  HAL_GPIO_Init(B1_GPIO_Port, &GPIO_InitStruct);

  /*Configure GPIO pins : PB0 PB1 PB2 PB10 */
  GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_10;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOB, &GPIO_InitStruct);

  /* USER CODE BEGIN MX_GPIO_Init_2 */

  /* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */

/* USER CODE END 4 */

/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return state */
    __disable_irq();
    while (1) {
    }
  /* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line number
  *         where the assert_param error has occurred.
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
  */
void assert_failed(uint8_t *file, uint32_t line)
{
  /* USER CODE BEGIN 6 */
  /* User can add his own implementation to report the file name and line number,
     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
```

```
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```

6 Visuals

This section will include one or more images of the final wiring of your system.

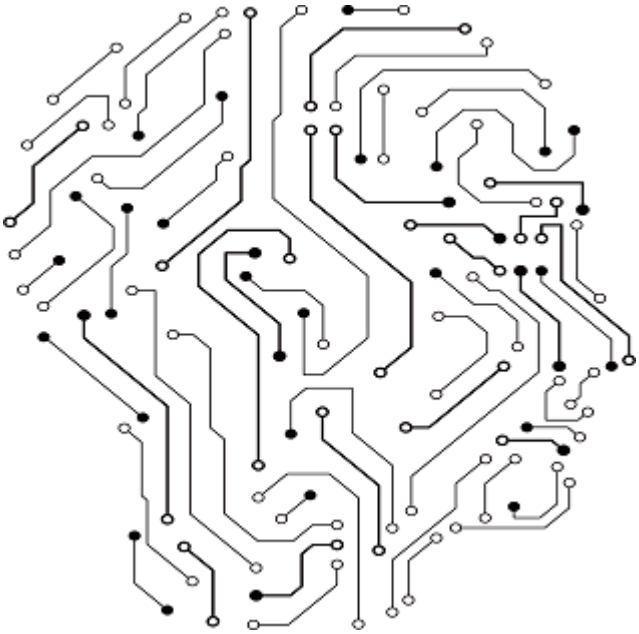

Figure 2: Replace this text with a caption for your image.)

7 Conclusion

This lab ended up coming together better than I expected. I was able to build a system that reads temperature and reacts to it in multiple ways using a servo, LEDs, and a buzzer. Seeing everything respond in real time helped things actually click instead of just feeling like random code. I also learned that wiring matters just as much as the code, since small mistakes caused very obvious problems. Overall, this lab helped me understand how sensors and outputs work together in a real system.

This project connects well with what we have been doing all semester because it combines a lot of separate ideas into one working design. Earlier labs focused on single things like ADC or timers, but this lab showed how they all depend on each other. It also made the idea of embedded systems feel more real, especially with a problem that could happen in real life like overheating electronics. Going forward, this lab makes me more confident working with hardware and gives me a better idea of how these systems could be expanded or used in actual products.

References